

PRN: 2124UCSM1057

Class: 3rd Year

Department: Cyber Security

ShopEase — 3-Tier Web Application: Shared Responsibility Mapping

Responsibility Mapping & Asset Definition — Unit 1

September 2026

Contents

1. Overview	2
Application Summary	2
Repository Structure	2
2. Architecture	3
Traffic & Component Flow	3
Tier Breakdown	4
2.1 Web Tier	4
2.2 Application Tier	4
2.3 Database Tier	4
2.4 Supporting Services	4
Network Segmentation	4
3. Shared Responsibility Matrix	5
Summary by Tier	7
4. Design Rationale	8
Why EC2 pushes more responsibility to the customer	8
Why RDS shifts responsibility to AWS	8
Why encryption is “Shared” rather than one-sided	8
Why network segmentation is entirely the customer’s job	8
Why DDoS protection is split into two rows	8
What this exercise demonstrates	9

1. Overview

This document covers the architecture and Shared Responsibility Matrix for **ShopEase**, a fictional e-commerce web application deployed on AWS using a classic 3-tier design (Web / App / DB).

The goal is to demonstrate understanding of the **AWS Shared Responsibility Model** by mapping it directly onto a specific, real architecture rather than repeating AWS's generic diagram.

Application Summary

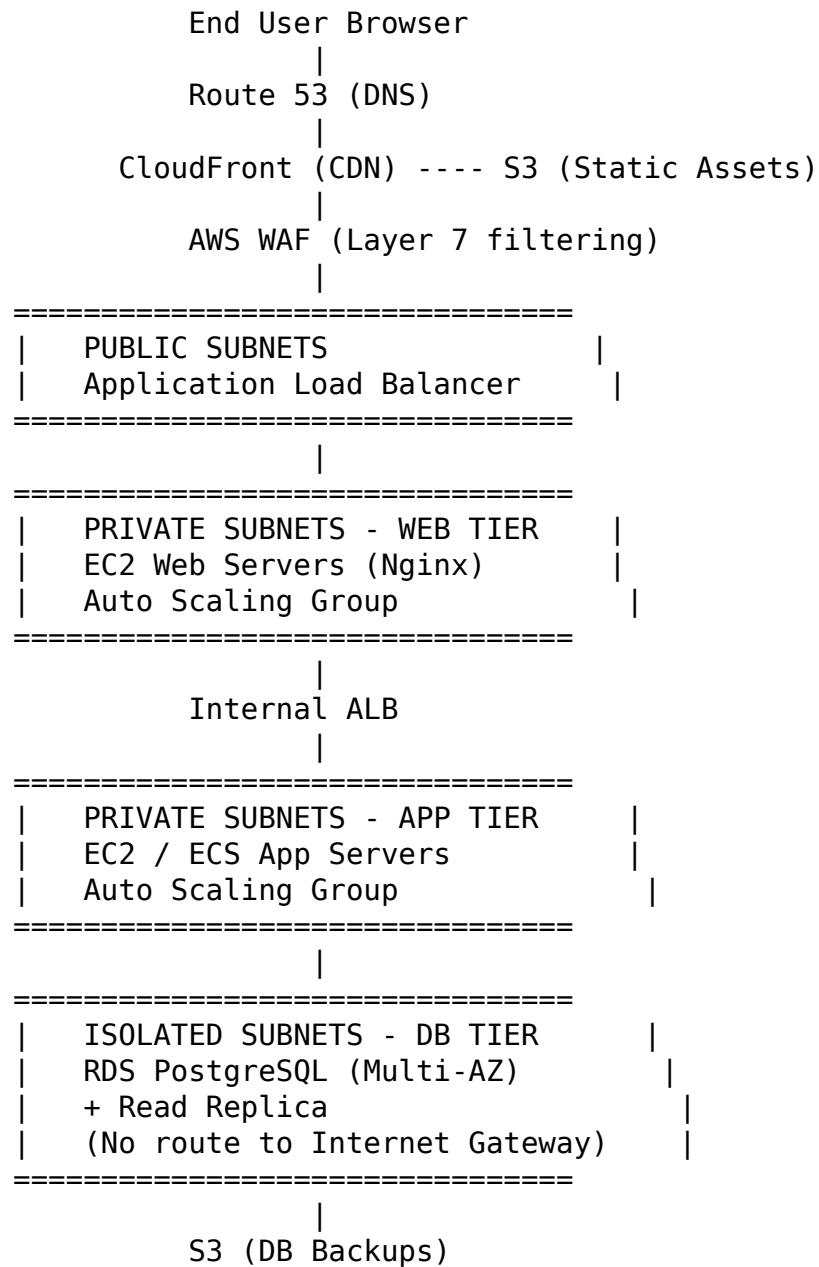
Attribute	Detail
Application	ShopEase (e-commerce web app)
Cloud Provider	AWS
Tiers	Web, Application, Database
Deployment Model	IaaS/PaaS hybrid (EC2 + managed RDS)

Repository Structure

```
shopease-shared-responsibility/  
├── README.md  
├── architecture.md  
├── shared-responsibility-matrix.md  
└── design-rationale.md
```

2. Architecture

Traffic & Component Flow



Cross-cutting services: KMS (encryption keys) | CloudWatch (logs/metrics)
| IAM (roles & policies) | NAT Gateway (outbound access for private subnets)

Tier Breakdown

2.1 Web Tier

- **Components:** CloudFront (CDN), AWS WAF, Application Load Balancer, EC2 Auto Scaling Group running Nginx
- **Placement:** Load balancer in public subnets; web servers in private subnets (not directly internet-facing)
- **Purpose:** Serves static content, TLS termination, request routing, DDoS/edge protection

2.2 Application Tier

- **Components:** Internal ALB, EC2 Auto Scaling Group (or ECS Fargate) running the application logic (Node.js/Java/Python API)
- **Placement:** Private subnets, only reachable from the web tier
- **Purpose:** Business logic, session handling, API endpoints, calls to database tier

2.3 Database Tier

- **Components:** Amazon RDS for PostgreSQL, Multi-AZ deployment with a read replica
- **Placement:** Isolated private subnets, no direct route to/from the internet
- **Purpose:** Persistent data storage (orders, users, inventory)

2.4 Supporting Services

Service	Role
Route 53	DNS management
CloudFront	Content delivery, edge caching
AWS WAF	Layer 7 filtering (SQLi, XSS rules)
S3	Static assets, log storage, DB backups
KMS	Encryption key management (EBS, RDS, S3)
CloudWatch	Logging, metrics, alarms
IAM	Identity, roles, least-privilege policies
NAT Gateway	Outbound internet access for private subnets

Network Segmentation

- **Public subnets:** ALB only
- **Private subnets (web/app):** No public IPs; traffic only via load balancers
- **Isolated subnets (DB):** No route to NAT/Internet Gateway at all

This segmentation is central to the responsibility matrix — the more AWS manages (e.g., RDS engine), the less the customer is responsible for at that layer, but network/access control design is always the customer's job.

3. Shared Responsibility Matrix

This matrix maps AWS's Shared Responsibility Model onto the specific components used in the ShopEase architecture. Responsibility is assessed **per component**, not as a single blanket statement for the whole app.

Legend: **AWS** = AWS's responsibility | **You** = Customer's responsibility | **Shared** = Both parties have a role

#	Control Area	Component(s)	Responsible	Notes
1	Physical data center security	All	AWS	Facilities, power, physical access controls
2	Hypervisor / host infrastructure	EC2 (web/app)	AWS	AWS secures and patches the virtualization layer
3	Global network infrastructure	All	AWS	Backbone routers, edge locations, core network
4	Guest OS patching	EC2 web & app servers	You	You choose the AMI and must patch the OS yourself
5	Web server software config/patching	Web tier	You	Nginx installed and maintained by customer on EC2
6	Application code security	App tier	You	Input validation, dependency updates, secure coding
7	Database engine patching	RDS PostgreSQL	AWS	Managed service — AWS applies engine patches
8	Database OS patching	RDS PostgreSQL	AWS	Underlying OS fully abstracted from customer

#	Control Area	Component(s)	Responsible	Notes
9	DB schema, queries, access control	RDS PostgreSQL	You	Table design, least-privilege DB users, query security
10	Network ACLs / Security Groups	VPC (all tiers)	You	Subnet segmentation and SG rules
11	VPC design & subnet isolation	VPC	You	Public/private/isolated subnet layout
12	IAM users, roles, policies	All	You	Least-privilege design, MFA, key rotation
13	Data encryption at rest	EBS, RDS, S3	Shared	AWS provides mechanism (KMS); you enable/manage keys
14	Data encryption in transit	ALB, CloudFront	Shared	AWS provides TLS support; you enforce HTTPS-only
15	DDoS protection (infrastructure)	CloudFront, ALB	AWS	Shield Standard is automatic and free
16	DDoS protection (application)	WAF rules	You	You write/tune rules for app-specific threats
17	Backup configuration & retention	RDS, S3	You	You configure retention windows, verify restores
18	Backup infrastructure durability	RDS, S3	AWS	AWS guarantees durability of stored backup data
19	Auto Scaling configuration	Web & App ASGs	You	Scaling policies, thresholds, health checks

#	Control Area	Component(s)	Responsible	Notes
20	Monitoring & alerting setup	CloudWatch	You	AWS provides service; you configure alarms/dashboards
21	Compliance certs of infrastructure	All	AWS	AWS maintains and provides audit reports
22	Compliance of the application (PCI-DSS scope)	App/DB tier	You	How you handle/store customer payment data
23	Client-side security	N/A	Neither	Outside both parties' control

Summary by Tier

Tier	AWS-Managed Share	Customer-Managed Share
Web (EC2-based)	Low (hardware, hypervisor, network backbone)	High (OS, web server, patching, config)
App (EC2/ECS-based)	Low-Medium	High (code, IAM, scaling, monitoring)
DB (RDS-managed)	High (OS, engine patching, HA infra)	Medium (schema, access control, encryption enablement)

Conclusion: Moving from unmanaged EC2 (web/app tiers) to a managed service (RDS) measurably shifts operational burden toward AWS — but network design, access control, encryption configuration, and data governance remain the customer's responsibility across every tier.

4. Design Rationale

Why EC2 pushes more responsibility to the customer

The web and app tiers run on **unmanaged EC2 instances**. AWS's responsibility stops at the hypervisor — everything above that (OS, installed software, patches, runtime, application code) is the customer's job. This is the core trade-off of IaaS: more control, more responsibility.

Design choice: EC2 was used for the web/app tiers instead of a fully-managed PaaS (like Elastic Beanstalk or Fargate-only) because the assignment calls for a "classic" 3-tier design where the responsibility split is easy to observe and explain. If ShopEase used ECS Fargate instead, the OS-patching row would shift from "You" to "AWS," since Fargate abstracts away the underlying host.

Why RDS shifts responsibility to AWS

RDS is a **managed service** — AWS handles the underlying OS, database engine installation, and patching. The customer only interacts with the database at the schema/query/access-control level. This is why the engine/OS patching rows are AWS's responsibility, while schema/access control remains ours — RDS doesn't stop us from writing an insecure schema or over-privileging a DB user.

Why encryption is "Shared" rather than one-sided

AWS provides the *mechanism* for encryption (KMS, TLS support on ALB/CloudFront), but does not enable it by default in most cases, nor does it choose key rotation policies. A customer who never turns on RDS encryption at rest has AWS's tooling available but has not fulfilled their side of the responsibility. This is a common real-world misconfiguration point, so it's called out explicitly as "Shared" rather than assumed.

Why network segmentation is entirely the customer's job

AWS provides the VPC primitive, but the actual **design decisions** — which subnets are public vs. private vs. isolated, what Security Group rules exist, how the DB tier is walled off from the internet — are 100% customer-driven. The DB subnet has no route to the Internet Gateway at all, a deliberate design choice, not something AWS enforces automatically.

Why DDoS protection is split into two rows

AWS Shield Standard automatically protects against common infrastructure-layer DDoS attacks (SYN floods, reflection attacks) with no configuration needed — that's AWS's responsibility. But protecting against **application-layer** abuse (e.g., credential stuffing against the login endpoint, scraping) requires the customer to write and tune WAF rules specific to ShopEase's own attack surface. Treating "DDoS protection" as a single row would obscure this distinction.

What this exercise demonstrates

The Shared Responsibility Model isn't a fixed line — it moves depending on **which AWS service you choose for each component**. The same application (ShopEase) has a different responsibility profile at the web tier (mostly ours) than at the database tier (mostly AWS's), purely because of the service choice (EC2 vs. RDS). Documenting this per-component, rather than as one blanket statement, is the actual point of the assignment.